1 Project Specification - AI-Enabled Predictive Maintenance Platform

---

1.1 Customer Background

A manufacturing company operates a fleet of industrial machines across multiple production sites. In recent years, the company has experienced increasing costs due to unplanned downtime and unexpected equipment failures. To improve operational efficiency and reduce maintenance costs, the company has invested in extensive sensor infrastructure that continuously captures machine operating data.

The company now wishes to leverage Artificial Intelligence (AI) and Machine Learning (ML) to transform this data into actionable insights. Specifically, the company seeks a predictive maintenance platform capable of validating sensor quality, predicting upcoming failures, and supporting maintenance planning decisions.

Your development organization has been contracted to design and implement this platform as a collection of independently developed services that together form a complete AI-enabled system.

The project is intended to simulate a realistic industrial AI engineering project. As such, success depends not only on building accurate machine learning models, but also on applying professional software engineering, deployment, testing, monitoring, and collaboration practices.

1.2 Project Overview

The system shall consist of three independently developed AI-enabled services:

- **C1: Sensor Quality and Data Validation Service**
- **C2: Failure Prediction Service**
- **C3: Maintenance Planning Service**

Each service will:

- Be developed by a separate team.
- Contain its own machine learning component.
- Expose functionality through REST APIs.
- Be independently deployable.
- Be integrated with the other services through agreed API contracts.

The complete platform should support the following high-level workflow:

1. Sensor and machine data enters the system.

2. Data quality issues are detected and handled.
3. Machine failure risk is predicted.
4. Maintenance recommendations are generated.
5. Model behaviour and concept drift are continuously monitored.

1.3 Dataset

You will receive a dataset consisting of operational and sensor data collected from **20 industrial machines over a period of three years**.

The dataset contains the following attributes:

- timestamp
- machine_id
- operating_mode
  - high load
  - normal load
  - idle
- temperature
- vibration
- pressure
- rpm
- power
- anomaly:
  - true
  - false
- anomaly_type
  - sensor spike
  - sensor frozen
  - missing value
- failure_probability_gt
- failure_in_7_days:
  - true
  - false
- remaining_useful_life_hours
- failure_type
  - bearing failure
  - cooling failure
  - pressure leak
  - motor imbalance
- important_features
- maintenance_action
  - replace bearing
  - repair cooling system
  - replace valve

  - align motor
- priority
  - high
  - medium
  - low
- concept_drift_active:
  - true
  - false

Note that some attributes are targets for the respective component, i.e., the ground truth. This ground truth can be used to train your models or for following components, it can be used to simulate a perfectly working previous component.

1.4 Data Usage Requirements

The dataset must be split chronologically to simulate a realistic deployment scenario.

1.4.1 Training and Validation

The first two years of data may be used for:

- model training
- feature engineering
- model selection
- validation

1.4.2 Testing

The final year of data must be reserved exclusively for testing.

No information from the testing period may be used during:

- model training
- hyperparameter tuning
- model selection

Teams must clearly document how the data was partitioned.

1.5 Team Organization

Each project consists of **11 students**. It means that there are **two groups** (11 students each group) working on the same project.

In each group, students will be divided into three teams:

Team	Service
Team A	Sensor Quality and Data Validation Service (C1)
Team B	Failure Prediction Service (C2)
Team C	Maintenance Planning Service (C3)

Each team typically consists of 3–4 students.

Each team is responsible for:

- requirements analysis
- architectural design
- implementation
- machine learning development
- deployment
- testing
- monitoring
- documentation

Each team maintains its own repository and develops its service independently.

The primary interaction between teams occurs through jointly defined REST APIs.

1.6 General Architecture Requirements

The system shall be implemented as a distributed architecture consisting of independently deployable services.

The architecture should support:

- clear separation of responsibilities
- independent deployment of services
- API-based communication
- maintainability
- scalability
- extensibility

REST APIs shall be used for communication between services.

The exact API design is the responsibility of the project teams.

Teams are expected to collaborate early to establish:

- API contracts
- request/response formats
- integration procedures
- versioning approaches

Synchronous REST communication is sufficient.

1.7 Service Requirements

1.7.1 C1: Sensor Quality and Data Validation Service

1.7.1.1 Business Objective Operational decisions should not be based on unreliable sensor measurements. The purpose of this service is to identify and manage sensor quality issues before data is used by downstream services.

1.7.1.2 Required Machine Learning Functionality This service must include a machine learning component that evaluates incoming sensor data and identifies potential quality problems.

The required capabilities are:

- Validate incoming sensor data
- Detect and impute missing values
- Detect corrupted measurements
- Detect unusual operating conditions and sensor drift
- Produce quality reports

The exact modelling approach is left to the team.

1.7.1.3 Required API Capabilities The service should provide functionality for:

Inputs: - POST chunks of data from different machines

Outputs: - POST to C2 to add cleaned data from one machine for fault prediction - GET quality report - GET model information

---

1.7.2 C2: Failure Prediction Service

1.7.2.1 Business Objective The company wishes to anticipate equipment failures before they occur and enable proactive maintenance.

1.7.2.2 Required Machine Learning Functionality This service must contain a machine learning model capable of predicting future machine failures.

The required capabilities are:

- Estimate failure probability
- Predict remaining useful life
- Provide explanation of prediction in form of feature importance

The modelling approach is chosen by the team.

1.7.2.3 Required API Capabilities

The service should provide functionality for:

Inputs: - POST chunks of cleaned data from one machine

Outputs: - POST to C3 to provide required information for predicting suitable maintenance action in case of predicted failure - GET prediction report - GET model information

---

1.7.3 C3: Maintenance Planning Service

1.7.3.1 Business Objective

Maintenance actions should be driven by machine condition and predicted failure risk rather than fixed schedules.

1.7.3.2 Required Machine Learning Functionality

This service must contain a machine learning component that recommends maintenance actions.

The required capabilities are:

- Recommend maintenance actions
- Provide explanation for recommendation

Examples include:

- replace bearing
- repair cooling system
- replace valve
- align motor

The service may incorporate outputs from the Failure Prediction Service.

1.7.3.3 Required API Capabilities

The service should provide functionality for:

Inputs: - POST machine information for machine predicted to fail

Outputs: - POST to provided endpoint maintenance recommendation and explanation - GET maintenance report - GET model information

---

1.8 Common Requirements for All Teams

Every team is expected to demonstrate professional AI engineering and software engineering practices.

1.8.1 Deployment

Each team must provide:

- Source code repository
- Automated build process
- Automated testing
- Deployment pipeline
- Deployable service

Docker-based deployment is recommended.

Teams may use technologies such as Kubernetes or cloud deployments if they believe these technologies add value, but they are not required.

1.8.2 Model Versioning

Machine learning models must be treated as maintainable software artefacts.

Each team must:

- maintain explicit model versions
- document changes between model versions
- identify which model version is deployed

The goal is to support traceability and reproducibility throughout the project.

1.8.3 Safeguards Against Model Failure

Real-world AI systems must be robust against model failures.

Each team must:

1.8.3.1 Identify Failure Scenarios

Examples include:

- missing data
- invalid input values
- abnormal sensor behaviour
- out-of-distribution data
- unavailable downstream services
- low-confidence predictions

1.8.3.2 Implement Fallback Behaviour

Examples include:

- rule-based alternatives
- default responses
- warning messages
- degraded operating modes

The implemented safeguards should be appropriate for the service and clearly documented.

1.8.4 Testing

Each service must include a comprehensive testing strategy.

At minimum, the following types of testing are required:

- Unit tests
- Integration tests
- API tests
- Data validation tests
- Accuracy checks

Testing should be incorporated into the automated development and deployment process whenever possible.

1.8.5 Monitoring and Concept Drift

All machine learning services require monitoring after deployment.

Each team must implement monitoring mechanisms appropriate for their service.

Monitoring should include:

- service health
- model behaviour
- prediction quality
- data quality

Teams must also develop a strategy for detecting and managing concept drift.

The dataset contains a concept drift indicator that may be used for evaluation and experimentation.

Concept drift monitoring and handling should be discussed and justified as part of the project.

1.9 Sprint Reviews

Participation in sprint reviews is mandatory.

No written submission is required.

Teams are expected to present and demonstrate their current progress.

1.9.1 Sprint Review 1

Each team must present:

- Requirements analysis
- System architecture
- API specification
- Initial machine learning prototype
- Initial deployment pipeline

The objective is to demonstrate that the project has a solid technical foundation and a feasible implementation plan.

1.9.2 Sprint Review 2

Each team must demonstrate:

- Working service
- Automated deployment
- Automated tests
- Model versioning
- Partial integration with other services

The objective is to verify progress toward a complete integrated solution.

1.10 Final Presentation

The final presentation consists of both team-level and project-level components.

1.10.1 Team Presentation

Each team must present:

- Requirements and design decisions
- Software architecture
- Machine learning solution
- Testing strategy
- Model versioning approach
- Failure handling strategy
- Concept drift strategy
- Deployment pipeline
- Lessons learned

The presentation should clearly explain both engineering decisions and machine learning decisions.

1.10.2 Project Demonstration

Each project must demonstrate the complete integrated system.

The demonstration should show:

1. Data entering the platform.
2. Sensor quality validation.
3. Failure prediction.
4. Maintenance planning.
5. Communication between services.
6. Deployment and operation of the integrated solution.

The focus is on demonstrating a complete AI-enabled system rather than three isolated machine learning models.

1.11 Assessment and Grading

Assessment is performed at both group and individual levels.

1.11.1 Group Requirements

To pass the project, a group must demonstrate:

- A working software component
- Meaningful machine learning functionality
- Automated deployment
- Testing and validation
- Monitoring capabilities
- Model versioning
- Collaboration across teams
- Professional software engineering practices

Failure to satisfy these requirements may result in the group not passing the project.

1.11.2 Individual Requirements

Each student must demonstrate:

- Active participation in the project

- Meaningful contribution to the group's work

Contributions must be documented through the project journal.

Examples of acceptable contributions include:

- software development
- machine learning development
- system architecture
- testing
- deployment
- integration
- documentation
- presentations

Insufficient contribution may result in an individual failing the project, even if the group project is successful.

1.12 Deliverables

Each team must provide:

- Source code repository
- Deployment pipeline
- Automated tests
- API documentation
- Trained machine learning model(s)
- Model versioning documentation
- Deployment instructions
- Final presentation materials

Each project must provide:

- Integrated system demonstration

1.13 Success Criteria

A successful project delivers a professional AI-enabled predictive maintenance platform in which:

- sensor quality is automatically assessed,
- machine failures can be predicted,
- maintenance actions can be recommended,
- machine learning models are deployed as operational services,
- models are monitored after deployment,
- concept drift is considered and managed,
- services communicate through well-defined APIs, and

- the complete solution can be demonstrated as a functioning integrated system.

The project should demonstrate not only machine learning competence, but also the software engineering and MLOps practices necessary to develop, deploy, maintain, and evolve AI-enabled systems in an industrial environment.